TOPIC 5.3

Overfitting and Underfitting

Why a model can be too simple, too flexible, or just right

WHAT YOU WILL LEARN

- Define **underfitting** and **overfitting** and spot each from training and test scores
- Explain why a low training error does **not** mean a good model
- Read a **training vs test error** curve and find the sweet spot
- Use a **train/test split** and **k-fold cross-validation** to check a model honestly
- List the main **remedies** for each problem

1. The goal: generalise

We train a model on the data we have, but we want it to work on data it has **never seen**. This ability is called **generalisation**. Two things can go wrong, and they are opposites.

DEFINITION **Underfitting**

The model is **too simple** to capture the pattern in the data. It performs badly on the training data **and** on new data. (*High bias.*)

DEFINITION **Overfitting**

The model is **too flexible**: it memorises the training data, including its random noise. It performs very well on the training data but badly on new data. (*High variance.*)

DEFINITION **Good fit**

The model captures the real pattern and ignores the noise. Training and test performance are both good and close to each other.

2. A picture of the three cases

Here the true pattern (dashed grey line) is a smooth wave, and we have only 20 noisy points (black dots). We fit polynomials of three different degrees. A degree-1 polynomial is a straight line; a degree-12 polynomial can bend 11 times.

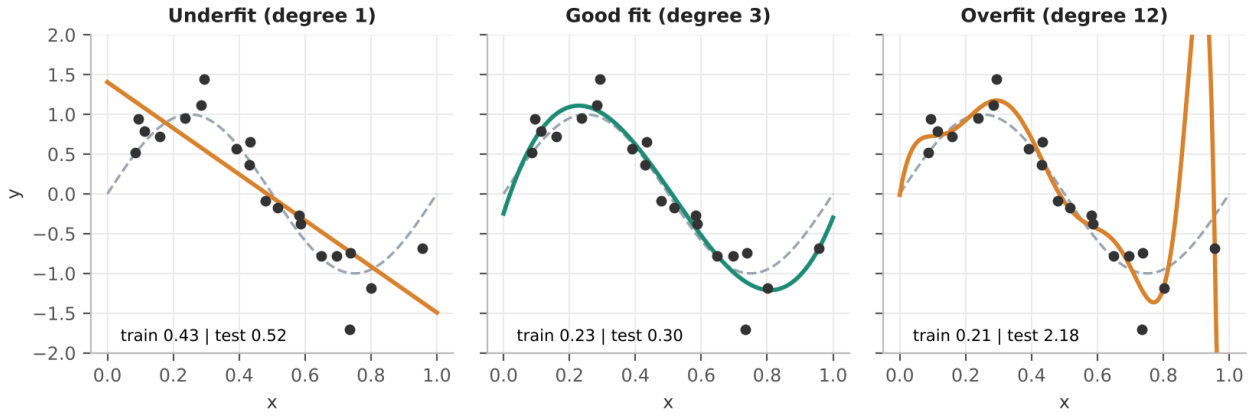


Figure 1. The same 20 points fitted with a line, a gentle curve and a very flexible curve. Errors are RMSE (smaller is better), measured on the 20 training points and on 300 fresh test points.

Model	Training error	Test error	Diagnosis
Degree 1 (straight line)	0.43	0.52	Underfit: both high
Degree 3	0.23	0.30	Good fit: both low and close
Degree 12	0.21	2.18	Overfit: training tiny, test much larger

The noise in this data has a standard deviation of 0.25, so even a perfect model cannot get below about 0.25 on test data. The degree-12 curve reaches a training error of 0.21, **below** that floor. That is a warning sign: it is fitting the noise, not the pattern.

3. Reading the error curve

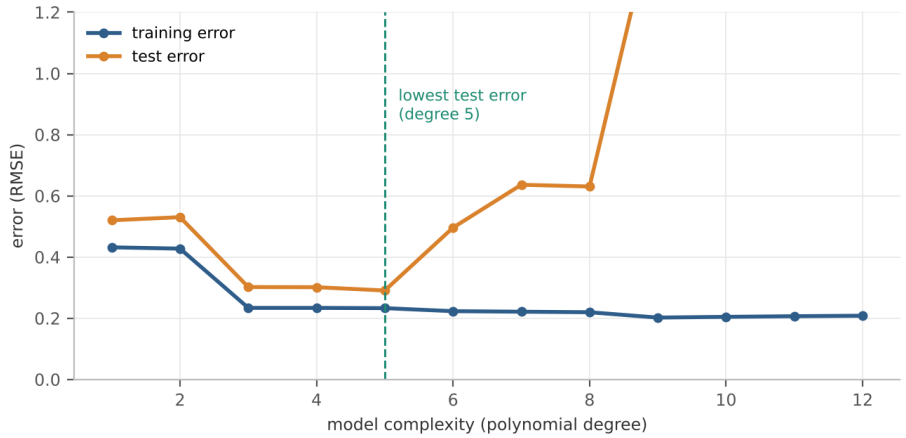


Figure 2. As the model becomes more complex, training error keeps falling, but test error falls and then rises again.

- **Training error** keeps going down as we add flexibility (more freedom lets the model fit the training points at least as well).
- **Test error** falls while the model is learning real structure, then rises when it starts memorising noise.
- The best model sits at the **bottom of the test curve**, not at the lowest training error.

> **WARNING** **A common mistake**
>
> Judging a model by its **training score**. A model that memorises the training data scores perfectly on it and still fails on anything new. Always report a score on data the model has not seen.

4. Diagnosing from the scores

Training score	Test score	What it usually means
Low	Low	**Underfitting**: the model is too simple or the features are weak
High	Much lower	**Overfitting**: the model memorised the training data
High	High, close to training	**Good fit**

5. Checking honestly: train/test split and cross-validation

Train/test split. Hide part of the data (often 20% to 30%) while training, then score the model on that hidden part. The result is an honest estimate of how it will do on new data.

k-fold cross-validation. One split can be lucky or unlucky. Instead, cut the data into **k equal folds** (commonly 5). Train on k − 1 folds, test on the remaining one, and repeat so that every fold is the test fold once. Average the k scores.

WORKED EXAMPLE **Cross-validation on the 20 points (5 folds)**

- Degree 1: average RMSE **0.49**
- Degree 3: average RMSE **0.26**
- Degree 12: average RMSE **56.82**
- Cross-validation picks degree 3 using the training data alone, without touching the test set

6. Fixing the problems

If the model is underfitting	If the model is overfitting
Use a more flexible model (higher degree, deeper tree)	Get **more training data**
Add informative features	Use a **simpler model** (lower degree, shallower tree)
Reduce regularization	Remove useless features (see 5.2)
Train longer (for iterative models)	Add **regularization**: Ridge, Lasso, Elastic Net (see 5.5)
	Use cross-validation to choose settings

Summary

SUMMARY

- **Underfitting**: too simple; poor on training **and** test data
- **Overfitting**: too flexible; excellent on training, poor on test data
- Training error keeps falling (or levels off) as complexity grows; **test error** has a minimum: the sweet spot
- Judge models on **unseen data**: train/test split, **k-fold cross-validation**
- Cure underfitting with more flexibility; cure overfitting with more data, simpler models and regularization

Practice questions

Try each question before reading the answer underneath it.

Q1. A model scores 99% on training data and 60% on test data. What is the problem?

Answer. **Overfitting**. It memorised the training data and does not generalise.

Q2. A model scores 55% on both. What is the problem?

Answer. **Underfitting**. It is too simple, so it fails everywhere. A more flexible model or better features may help.

Q3. Why can't we pick the model with the lowest training error?

Answer. The most flexible model always wins on training error, even when it is just memorising noise. Only unseen data shows true performance.

Q4. What does 5-fold cross-validation do?

Answer. It splits the data into 5 folds, trains on 4 and tests on 1, rotating so that each fold is tested once, then averages the 5 scores.

Q5. Name three ways to reduce overfitting.

Answer. More training data, a simpler model, and regularization (also removing useless features).